Projet noté — Site vitrine d'un événement

Module CSS avancé · Licence 3 · Projet en groupe de trois (trinôme)

Calendrier	Lancement : **J1 16h00** · Rendu : **J3 14h30** · Soutenances : **J3 à partir de 14h45**
Évaluation	**/20** = **18 pts de groupe** (critères A à G) + **2 pts individuels** (question de soutenance, critère H) — barème détaillé : `Bareme.md`
Squelette de départ	dossier `starter/`

Objectifs pédagogiques

À l'issue du projet, vous devez être capable de :

- **concevoir une feuille de styles maintenable** : couches (`@layer`), design tokens, nesting natif, convention de nommage ;
- **réaliser une mise en page responsive mobile-first** avec Grid, `subgrid`, Flexbox et container queries ;
- **mettre en œuvre un thème clair/sombre**, des animations qui respectent `prefers-reduced-motion` et les bases de l'accessibilité ;
- **justifier oralement vos choix techniques** et expliquer n'importe quelle partie du code de votre groupe.

---

1. Le brief du client

Dans un cadre fictif, vous intervenez pour l'agence web *Studio Cascade*, à qui un organisateur d'événement commande un site vitrine. Le client exprime les besoins suivants :

- une page d'accueil attractive, parfaitement utilisable **sur téléphone** (les visiteurs achètent leurs billets en mobilité) ;
- un **mode sombre** ;
- un site **accessible à tous** ;
- du **CSS propre, organisé et documenté**, que son équipe pourra maintenir (le site précédent, réalisé avec un framework, s'est révélé impossible à maintenir).

Vous disposez de trois jours pour livrer ce site vitrine, puis vous présentez et justifiez vos choix lors d'une **soutenance** devant l'enseignant.

---

2. Choisir son thème (J1, 16h00)

Chaque groupe choisit **un événement fictif** parmi les thèmes suivants (un nom d'événement et une identité visuelle propres sont attendus) :

Thème	Exemple de client fictif	Pistes visuelles
Festival de musique	« Les Nuits du Beffroi », 3 scènes, 2 jours	affiches sérigraphiées, couleurs néon
Salon du jeu vidéo	« Pixel Expo », stands et tournois	pixel art, interfaces de jeu
Conférence tech	« DevFlex Conf », talks et ateliers	éditorial, code, monospace
Exposition d'art	« Chromatique », parcours d'œuvres	galerie, grands blancs, typographie forte
Marché de créateurs	« Le Marché des Petites Mains »	artisanal, textures, couleurs chaudes
Compétition e-sport	« Grid Masters Cup », phases de poules	sombre, contrasté, énergique
Thème libre	à faire **valider par l'enseignant**	—

Règle : **deux groupes au maximum** par thème, et dans ce cas avec des identités visuelles nettement différentes. Les thèmes sont attribués dans l'ordre des demandes.

---

3. Contenu attendu

Une **page d'accueil complète** (`index.html`). Une 2e page est **facultative** (ex. page détaillée d'un intervenant) et n'apporte pas de points en soi, mais peut servir de terrain aux View Transitions.

Le contenu est **fictif mais réaliste** : pas de « lorem ipsum » dans le rendu final.

Section	Contenu minimal attendu
Hero	Nom de l'événement (seul `h1` de la page), date(s), lieu, accroche, appel à l'action vers la billetterie. Visuel ou décor CSS.
Programme en grille	Au moins **6 créneaux** répartis sur **2 jours ou 2 scènes/salles** (horaire, titre, lieu/scène). Mise en page en Grid (c'est un bon candidat pour `grid-template-areas` et/ou `subgrid`).
Intervenants / artistes en cartes	Au moins **6 cartes** : image (ou illustration), nom, rôle, courte bio, lien. Les cartes doivent rester alignées même avec des textes de longueurs différentes (`subgrid`).
Billetterie	Au moins **3 offres** (ex. Pass 1 jour, Pass week-end, Pass VIP) avec prix, contenu de l'offre et bouton. Une offre **mise en avant**.
FAQ	Au moins **5 questions** en `<details>` / `<summary>`.
Footer	Contact, liens réseaux (fictifs), mentions « événement fictif — projet étudiant », **crédits des images et polices**.

Plus une **en-tête avec navigation** vers les sections (ancres).

---

4. Exigences techniques

Chaque exigence doit être **localisée dans votre `README.md`** (fichier + ligne, cf. `starter/README_rendu.md`). Une exigence introuvable est considérée comme absente.

Exigence 1 — Architecture

- Ordre des layers déclaré **une seule fois** en tête de `css/main.css` : `@layer reset, base, layout, components, utilities;`

- Chaque fichier CSS enveloppé dans son layer (le starter le fait déjà).
- **Design tokens** en custom properties : des **primitifs** (palette, échelle d'espacements, rayons, durées) et des **sémantiques** (rôle : `--color-surface`, `--color-text`, `--color-accent`…). Les composants n'utilisent **que** les tokens sémantiques.
- **Nesting natif** (sans préprocesseur), ≤ 3 niveaux.
- **Nommage cohérent** et annoncé dans le README (ex. BEM : `.card`, `.card__title`, `.card--featured`).

Accepté	Non accepté
`.card { padding: var(--space-card); &:hover { … } }` dans `components.css`, à l'intérieur de `@layer components`	Un `style.css` unique de 900 lignes sans layer
`--color-accent: var(--color-violet-600);` puis `.btn { background: var(--color-accent); }`	`.btn { background: #7a3cff; }` répété dans 12 règles
`.u-visually-hidden` dans `utilities`, qui gagne grâce à l'ordre des layers	`.titre-rouge-2`, `#bloc3 div div p`

Exigence 2 — Mise en page

- Au moins **une grille avec `grid-template-areas`** (≥ 3 zones) qui se réorganise entre mobile et desktop.
- Au moins **un `subgrid`** à l'effet visible (alignement des contenus de cartes ou des créneaux du programme).
- **Flexbox pour les composants** (navigation, groupes de boutons, en-têtes de cartes…) avec `gap`.

Accepté	Non accepté
`.programme { grid-template-areas: "titre" "jour1" "jour2"; }` puis, en desktop, `"titre titre" "jour1 jour2"`	Des `float`, ou une mise en page « au pixel » en `position: absolute`
`.card { grid-row: span 4; display: grid; grid-template-rows: subgrid; }` dans une grille parente	`grid-template-rows: subgrid` sur un élément qui n'occupe qu'une seule ligne (aucun effet)
`.nav__list { display: flex; flex-wrap: wrap; gap: var(--space-3); }`	Des `margin-left: 37px` pour espacer des liens

Exigence 3 — Responsive

- **Mobile-first** : styles de base pour petit écran, puis `@media (width >= 48rem)` (ou `min-width`) pour élargir.
- **Typographie fluide** avec `clamp()` (titres et corps, ou espacements de section), idéalement dans les tokens.
- Au moins **un composant piloté par container query**, utilisé dans **deux contextes de largeurs différentes**.
- **Aucun défilement horizontal à 320 px.**

Accepté	Non accepté
`--font-size-h1: clamp(2rem, 1.2rem + 4vw, 4.5rem);`	`h1 { font-size: 64px; }` puis 5 media queries en `max-width`

Accepté	Non accepté
`.speakers { container: speakers / inline-size; }` + `@container speakers (inline-size > 40rem) { .card { … } }`, la même carte apparaissant en grille et dans un bandeau étroit	Une container query sur un conteneur qui fait toujours 100 % de la fenêtre (elle se comporte comme une media query)
`img { max-inline-size: 100%; block-size: auto; }`	Une barre de défilement horizontale à 320 px à cause d'un tableau ou d'une image de 600 px

Exigence 4 — Sélecteurs modernes

- Au moins un usage **pertinent** de `:has()`.
- Usage de `:is()` et/ou `:where()` pour factoriser ou maîtriser la spécificité.

Accepté	Non accepté
`.offer:has(.offer__badge) { … }` : l'offre qui contient un badge est mise en avant automatiquement	`body:has(main) { … }` (toujours vrai, n'apporte rien)
`:where(h1, h2, h3) { text-wrap: balance; }` dans `base` (spécificité nulle, facile à surcharger)	Un `:is()` qui ne contient qu'un seul sélecteur, sans raison

Exigence 5 — Thème clair/sombre

- Thème sombre via `light-dark()` (avec `color-scheme: light dark`) ou `@media (prefers-color-scheme: dark)`.
- Les **deux thèmes** sont complets et respectent les contrastes AA.
- **Bonus technique** : bascule CSS-only (case à cocher + `:root:has(#theme-toggle:checked) { color-scheme: dark; }`).

Accepté	Non accepté
`--color-surface: light-dark(var(--color-sand-50), var(--color-night-900));`	Un thème sombre où les cartes restent blanches ou le texte gris sur gris
Bascule `:has()` qui modifie `color-scheme`, donc tous les `light-dark()` d'un coup	Un `filter: invert(1)` sur toute la page

Exigence 6 — Animations

- Au moins **2 transitions** utiles, sur des propriétés précises (de préférence `transform`, `opacity`, couleurs).
- Au moins **1** **`@keyframes`** utilisé de façon pertinente.
- Respect de **`prefers-reduced-motion`** : toutes les animations de mouvement supprimées ou réduites.
- **Bonus technique** : animation pilotée par le scroll (`animation-timeline: view()`) **dans un** **`@supports`** — elle n'est **pas prise en charge par Firefox**, la page doit donc rester correcte sans elle ; View Transitions ; `@starting-style` / popover.

Accepté	Non accepté
`.card { transition: translate var(--duration-fast) var(--ease-out); &:hover { translate: 0 -4px; } }`	`transition: all 1s`

Accepté	Non accepté
`@keyframes pop-in { from { opacity: 0; scale: .9; } }` sur l'apparition du hero	Une animation infinie qui clignote en permanence à côté du texte
`@media (prefers-reduced-motion: reduce) { … }` qui neutralise **toutes** les animations sans laisser d'élément invisible	Un élément démarré à `opacity: 0` qui reste invisible quand l'animation est désactivée
`@supports (animation-timeline: view()) { … }`	`animation-timeline` sans `@supports`, qui casse l'affichage dans Firefox

Exigence 7 — Accessibilité

- **HTML sémantique** : `header`, `nav`, `main`, `section` titrées, `footer` ; un seul `h1` ; hiérarchie de titres continue ; `lang="fr"`.
- `:focus-visible` **visible** sur tous les éléments interactifs ; page entièrement utilisable au clavier.
- **Contrastes AA** dans les deux thèmes (4,5:1 texte courant, 3:1 grand texte et composants d'interface).
- **Images avec** `alt` pertinent (`alt=""` pour les images décoratives).

Accepté	Non accepté
`:focus-visible { outline: 3px solid var(--color-focus); outline-offset: 3px; }`	`*:focus { outline: none; }` sans remplacement
`<details><summary>Puis-je revendre mon billet ?</summary>… </details>`	Une FAQ en `<div onclick>`
`<img src="img/dj.webp" alt="DJ Nova aux platines, sous des lumières violettes">`	`alt="image"`, `alt="photo1.jpg"` ou pas d'`alt`

Exigence 8 — Qualité

- CSS **valide** au validateur W3C (https://jigsaw.w3.org/css-validator/). Le validateur ne reconnaît pas toujours les syntaxes les plus récentes : listez ces avertissements dans le README, ils ne sont pas pénalisés.
- Code **commenté** : rôle de chaque fichier, sections, raison des choix non évidents.
- **Pas de** `!important`, sauf dans un utilitaire dont l'usage est justifié en commentaire.

Accepté	Non accepté
`/* Le hero passe en 2 colonnes : le visuel prend la zone "media". */`	`/* css */`, `/* style */` ou aucun commentaire
Aucune `!important` : les conflits sont réglés par les layers	`color: red !important;` pour « gagner » contre une autre règle

---

5. Contraintes

- **CSS pur** : pas de framework CSS (Bootstrap, Tailwind…), pas de préprocesseur (Sass, Less, PostCSS), pas de CSS-in-JS, pas de gabarit récupéré sur Internet.
- **JavaScript interdit**, sauf **≤ 20 lignes** (hors lignes vides et commentaires), dans un seul fichier `js/main.js`, **chaque usage justifié** dans le README. Exemple acceptable :

`document.startViewTransition()` pour animer un changement, ou mémoriser le thème choisi. Non acceptable : JavaScript pour ouvrir la FAQ (`<details>` le fait), pour un menu (Popover API, `<details>` ou `:has()` le font), ou pour la mise en page. Aucune bibliothèque JS.

- **Pas de build** : la page s'ouvre directement dans le navigateur.
- Navigateur de référence : Chrome/Edge récent ; **testez aussi dans Firefox** (replis, `@supports`).
- Les fonctionnalités utilisées dans les exigences (`@layer`, nesting, `subgrid`, container queries, `:has()`, `light-dark()`) sont **Baseline** : utilisables sans repli dans les navigateurs récents. Les fonctionnalités valorisées en bonus technique (scroll-driven animations, anchor positioning, View Transitions…) s'écrivent en **amélioration progressive**.
- Images : libres de droits (Unsplash, Pexels…), générées par vous, ou illustrations CSS/SVG. **Créditez-les** dans le footer. Poids raisonnable (WebP/AVIF conseillé).

---

6. Jalons

Quand	Jalon	Ce qui est attendu
J1 · 16h00	Lancement + **wireframe validé**	Thème choisi, nom de l'événement, wireframe papier **mobile + desktop** de la page (zones de la grille nommées), répartition des rôles. À faire **valider par l'enseignant avant 16h30**.
J2 · 13h30	**Point d'avancement** (stand-up, 2 min par groupe)	Fait / à faire / blocage. Le squelette HTML complet et les tokens doivent exister.
J2 · 15h15	**Revue croisée**	Évaluation par les pairs, non notée : chaque groupe évalue le site d'un autre groupe avec `Grille_revue_croisee.md`, puis récupère la grille qui le concerne et corrige.
J3 · 14h30	**Rendu**	Dépôt final (cf. §7). Au-delà : −1 point par tranche de 15 min commencée.
J3 · 14h45	**Soutenances** — 7 min par groupe	4 min de démonstration (parcours de la page, mobile → desktop, thème sombre, clavier) + 3 min de **questions individuelles** (une par membre). L'ordre de passage est fixé par l'enseignant.

---

7. Modalités de rendu

Au choix :

1. **Dépôt Git** (GitHub, GitLab, forge de l'IUT) : lien envoyé à l'enseignant dès J2 ; le **dernier commit avant 14h30** fait foi (vous pouvez ajouter un tag `rendu`). Chaque membre committe avec son propre compte : l'historique sert à la modulation individuelle.
2. **Archive** nommée **`groupe-NOM.zip`** (NOM = nom du groupe déclaré à l'enseignant, en minuscules, sans espaces ni accents, ex. `groupe-nuits-du-beffroi.zip`), déposée sur l'ENT avant 14h30.

Contenu attendu :

```
groupe-NOM/
  index.html
  README.md          ← README_rendu.md complété (renommé)
  css/  main.css, reset.css, tokens.css, base.css, layout.css, components.css, utilities.css
  img/               (images optimisées)
  js/main.js         (facultatif, ≤ 20 lignes)
  page2.html         (facultatif)
```

Le `README.md` est obligatoire (absent → −1) : il localise chaque exigence (fichier:ligne), annonce votre convention de nommage, liste les éventuels avertissements du validateur, justifie le JavaScript, déclare l'usage d'outils d'IA et indique qui a fait quoi.

---

8. Organisation conseillée dans le trinôme

Trois rôles « référents », qui **ne dispensent personne de comprendre tout le code** (la question individuelle peut porter sur n'importe quelle partie) :

Rôle	Responsabilités principales	Exigences surtout concernées
Référent·e architecture	Layers, tokens, conventions de nommage, relecture, validation W3C, README	1, 8
Référent·e mise en page	Grilles, subgrid, Flexbox, responsive, container query, test à 320 px	2, 3
Référent·e thème, animations et accessibilité	Thème clair/sombre, animations, `:has()`, accessibilité (focus, contrastes)	4, 5, 6, 7

Conseils :

- Travaillez **chacun dans son fichier** (`layout.css`, `components.css`…) pour limiter les conflits Git.
- Faites une **relecture croisée interne** : chaque membre explique à un autre une partie qu'il n'a pas écrite (c'est le meilleur entraînement à la soutenance).
- Tournez les rôles au J3 matin si un membre n'a pas touché à une notion.

---

9. FAQ étudiants

Peut-on utiliser Google Fonts ? Oui, **via un `<link>` dans le `<head>`** (avec `display=swap`). Limitez-vous à 2 familles. Créditez-les dans le footer. Les polices système sont aussi un très bon choix.

Peut-on utiliser ChatGPT ou un autre outil d'IA ? Oui, **seulement si c'est déclaré et maîtrisé** : indiquez dans le README quels outils, pour quoi faire (ex. « générer les textes fictifs des intervenants », « expliquer une erreur de subgrid »). Chaque membre doit pouvoir **expliquer chaque ligne** du code en soutenance : c'est ce que vérifie la **question individuelle (critère H)**. Un code que vous ne savez pas expliquer vous fait perdre les points du critère H et ne rapporte aucun bonus technique, quelle que soit sa qualité.

Peut-on réutiliser le code des TP ? Oui, c'est votre code : adaptez-le à votre thème. Pas de gabarit ni de composant récupéré tel quel sur Internet.

Peut-on utiliser des icônes ? Oui : SVG en ligne ou fichiers SVG (créditez la source). Pas de bibliothèque d'icônes en JavaScript.

Le validateur W3C signale `light-dark()` ou le nesting comme erreurs, que faire ? Le validateur a parfois un temps de retard sur les nouveautés. Listez ces lignes dans la section prévue du README : elles ne sont pas pénalisées. Les vraies erreurs (accolade manquante, propriété mal orthographiée) le sont.

Le menu burger doit-il être en JavaScript ? Non : utilisez la Popover API (`popover` + `popovertarget`), un `<details>` ou une case à cocher + `:has()`. Sur desktop, la navigation peut simplement s'afficher en ligne.

Un membre du groupe est absent une demi-journée, que se passe-t-il ? Prévenez l'enseignant. La note de groupe reste commune ; la modulation individuelle (±2) tient compte de la contribution réelle observée (historique Git, sprints).

Faut-il une deuxième page ? Non, elle est facultative. Une page d'accueil complète et soignée vaut mieux que deux pages bâclées.

Où trouver le barème ? Dans `Bareme.md` : chaque sous-critère y est décrit avec ce qui vaut 0, partiel ou complet. Servez-vous-en comme liste de vérification avant le rendu.